OKLUT HRMS

Complete API Reference & Data Interface Specification

Project Architecture: React 18 SPA + Supabase PostgreSQL + Edge Functions + SMTP Gateway

Document Version: 1.0.0 | Production URL: https://hrm.oklut.com | Date: September 2026

1. Architecture & Integration Overview

This document provides an exhaustive reference of all APIs, services, and database interfaces implemented in the OKLUT Human Resource Management System (HRMS). The application connects directly to Supabase via TypeScript data client layers with Row-Level Security (RLS) enforcement, alongside custom HTTP endpoints for background SMTP dispatch and edge compute.
Total API interfaces documented: 64 unique methods spanning 12 business domains.

2. Authentication & User Session Management

Source: src/features/auth/ & public.users

API / Function	Table / Target	Operation	Parameters	Return Type	Description & Business Logic
login	public.users, employees	**SELECT / AUTH**	email, password	*Promise<UserSession>*	Validates user credentials, verifies active status (blocks deactivated users), resolves role and employee profile.
logout	Client Session	**STATE CLEAR**	None	*void*	Clears active JWT tokens, resets local application state and cached employee permissions.
forgotPassword	audit_logs, users	**SELECT + INSERT**	email: string	*Promise<{ success }>*	Verifies email existence, generates secure reset token, and queues a password reset email into audit_logs.
resetPassword	public.users	**UPDATE**	token, newPassword	*Promise<{ success }>*	Validates token expiry, updates password hash in public.users, and revokes prior session tokens.

3. Employee Workforce Management

Source: src/lib/api/employees.ts

API / Function	Table / Target	Operation	Parameters	Return Type	Description & Business Logic
fetchEmployees	public.employees	**SELECT**	None	*Promise<Employee[]>*	Retrieves all employees with joined department, designation, and user account status.
fetchEmployee	public.employees	**SELECT**	id: string	*Promise<Employee>*	Fetches single employee details including personal, bank, statutory, and salary profile.
createEmployee	employees, users	**INSERT (Transaction)**	CreateEmployeePayload	*Promise<Employee>*	Generates employee code (EMP-XXX), creates record in employees, and creates login account in public.users.
updateEmployee	public.employees	**UPDATE**	id: string, Partial<Employee>	*Promise<Employee>*	Updates employee details including department, designation, salary profile, and personal records.
deleteEmployee	public.employees	**DELETE / SOFT**	id: string	*Promise<void>*	Deactivates user login and removes employee from active directory.
deleteEmployeeByIdOrCode	employees, users	**DELETE**	identifier: string	*Promise<void>*	Deletes employee by either primary key UUID or company employee code (EMP-XXX).

4. Attendance, Time Tracking & Real-Time Clocks

Source: src/lib/api/attendance.ts

API / Function	Table / Target	Operation	Parameters	Return Type	Description & Business Logic
fetchTodayAttendance	public.attendance	**SELECT**	employeeId: string	*Promise<Attendance>*	Fetches current day punch status (check_in, check_out, total_hours, break_duration).
fetchAttendanceMonth	public.attendance	**SELECT**	employeeId, year, month	*Promise<Attendance[]>*	Queries 30-day attendance record for monthly payroll calculation and calendar visualization.
fetchAttendanceLog	public.attendance	**SELECT**	employeeId, start, end	*Promise<Attendance[]>*	Fetches filtered date-range attendance logs for manager reporting and audit verification.
checkIn	public.attendance	**INSERT**	employeeId, location?	*Promise<Attendance>*	Records daily punch-in timestamp, location

API / Function	Table / Target	Operation	Parameters	Return Type	Description & Business Logic
					coordinates, and initializes shift tracking.
checkOut	public.attendance	**UPDATE**	employeeId: string	*Promise<Attendance>*	Records punch-out time, computes total worked duration, and updates shift status.
setBreak	public.attendance	**UPDATE**	employeeId, isBreak	*Promise<Attendance>*	Toggles active meal or rest break, tracking cumulative pause duration during shift.

5. Leave & Holiday Administration

Source: src/lib/api/attendance.ts

API / Function	Table / Target	Operation	Parameters	Return Type	Description & Business Logic
fetchLeaveTypes	public.leave_types	**SELECT**	None	*Promise<LeaveType[]>*	Retrieves leave categories (Casual, Sick, Earned, Maternity) and annual quota allocations.
fetchLeaveRequests	public.leave_requests	**SELECT**	employeeId?: string	*Promise<LeaveRequest[]>*	Fetches submitted leave requests with employee details, duration, status, and approval notes.
applyLeave	public.leave_requests	**INSERT**	LeaveRequestPayload	*Promise<LeaveRequest>*	Submits new time-off application; validates sufficient remaining balance in leave_balances.
reviewLeave	leave_requests, balances	**UPDATE (Transaction)**	requestId, status, note	*Promise<LeaveRequest>*	Approves or rejects request; upon approval, automatically deducts days from leave_balances.
cancelLeave	leave_requests, balances	**UPDATE**	requestId: string	*Promise<void>*	Cancels pending or approved request and restores deducted days back to leave_balances.
fetchLeaveBalances	public.leave_balances	**SELECT**	employeeId, year	*Promise<LeaveBalance[]>*	Retrieves total, used, and remaining day allowances across all leave types for specified year.
fetchHolidays	public.holidays	**SELECT**	year?: number	*Promise<Holiday[]>*	Lists gazetted company holidays and optional floating observance days.
createHoliday	public.holidays	**INSERT**	HolidayPayload	*Promise<Holiday>*	Adds new holiday date and title to company operational calendar.
deleteHoliday	public.holidays	**DELETE**	id: string	*Promise<void>*	Removes designated holiday from public schedule.

6. Candidate Portal & Applicant Self-Service

Source: src/lib/api/candidatePortal.ts

API / Function	Table / Target	Operation	Parameters	Return Type	Description & Business Logic
candidateLogin	public.candidates	**SELECT**	referenceId, dateOfBirth	*Promise<CandidateProfile>*	Authenticates candidate via Reference ID (e.g., CAND-XXXXXX) and Date of Birth password.
fetchCandidatePortal	candidates, interviews, offers	**SELECT (Multi)**	referenceId: string	*Promise<CandidatePortalData>*	Loads applicant application progress, scheduled interview rounds, exam link, and offer letters.
bookInterviewSlot	interviews, candidates	**UPDATE**	slotId, candidateId	*Promise<InterviewBooking>*	Reserves candidate interview timeslot and generates calendar confirmation entry.
cancelInterviewSlot	public.interviews	**UPDATE**	bookingId: string	*Promise<void>*	Releases booked interview slot back to pool of open recruiter availability.
submitRescheduleRequest	interviews, audit_logs	**UPDATE + INSERT**	bookingId, reason, slots	*Promise<void>*	Flags interview for reschedule and notifies hiring team with preferred time windows.
revertRescheduleRequest	public.interviews	**UPDATE**	bookingId: string	*Promise<void>*	Reverts pending reschedule request and retains original meeting schedule.
startExam	candidates, audit_logs	**UPDATE**	examId, candidateId	*Promise<ExamSession>*	Initializes online assessment session and records countdown start timestamp.
submitExam	candidates, audit_logs	**UPDATE**	examId, answers	*Promise<ExamResult>*	Scores submitted exam answers, records ATS marks, and updates candidate stage.
acceptOfferTerms	public.offers	**UPDATE**	candidateId, terms	*Promise<Offer>*	Records candidate agreement to employment bond, relocation, and background check terms.
respondToOffer	offers, candidates	**UPDATE**	candidateId, status, note	*Promise<Offer>*	Candidate formally accepts or declines job offer letter; updates onboarding pipeline.
disqualifyCandidate	candidates, audit_logs	**UPDATE**	candidateId, reason	*Promise<void>*	Marks candidate as Rejected with rejection code and logs action for talent compliance.

7. Recruitment Pipeline & ATS Management

Source: src/lib/api/modules.ts

API / Function	Table / Target	Operation	Parameters	Return Type	Description & Business Logic
fetchJobOpenings	public.job_openings	**SELECT**	None	*Promise<JobOpening[]>*	Fetches open job positions with department, required skills, vacancy count, and status.
createJobOpening	public.job_openings	**INSERT**	JobOpeningPayload	*Promise<JobOpening>*	Publishes new job requisition visible on internal dashboard and external /careers portal.
updateJobOpening	public.job_openings	**UPDATE**	id, Partial<JobOpening>	*Promise<JobOpening>*	Edits job description, closing date, salary band, or toggles status (Active/Closed).
deleteJobOpening	public.job_openings	**DELETE**	id: string	*Promise<void>*	Removes requisition and archives associated applicant submissions.
fetchCandidates	public.candidates	**SELECT**	jobOpeningId?: string	*Promise<Candidate[]>*	Queries all applicants across pipeline stages (Applied, Screening, Interview, Offered).
createCandidate	public.candidates	**INSERT**	CandidatePayload	*Promise<Candidate>*	Creates candidate profile, assigns unique reference ID, and sets ATS initial score.
updateCandidateStatus	public.candidates	**UPDATE**	id, status	*Promise<Candidate>*	Updates hiring status (Under Review, Selected, Rejected, Hired).
updateCandidateStage	public.candidates	**UPDATE**	id, stage	*Promise<Candidate>*	Transitions candidate across workflow stages (Screening, Round 1, Round 2, HR Round).
fetchInterviews	public.interviews	**SELECT**	candidateId?: string	*Promise<Interview[]>*	Queries scheduled interview rounds with meeting links, interviewers, and feedback.
createInterview	public.interviews	**INSERT**	InterviewPayload	*Promise<Interview>*	Schedules interview round, meeting link, date/time, and assigns interviewer.
updateInterviewStatus	public.interviews	**UPDATE**	id, status, feedback?	*Promise<Interview>*	Records round completion status (Passed, Failed, Rescheduled) with reviewer remarks.
fetchOffers	public.offers	**SELECT**	candidateId?: string	*Promise<Offer[]>*	Fetches formal offer records with CTC, joining dates, designation, and signing status.
createOffer	public.offers	**INSERT**	OfferPayload	*Promise<Offer>*	Issues offer letter to candidate and activates offer review in candidate portal.
updateOfferStatus	public.offers	**UPDATE**	id, status	*Promise<Offer>*	Updates offer acceptance status (Draft, Sent, Accepted, Declined).

8. Payroll & Statutory Compensation Engine

Source: src/lib/api/core.ts

API / Function	Table / Target	Operation	Parameters	Return Type	Description & Business Logic
fetchPayrollProfiles	public.payroll_profiles	**SELECT**	employeeId?: string	*Promise<PayrollProfile[]>*	Fetches salary structures (Basic, HRA, Allowances, PF, ESI, Professional Tax, TDS).
fetchPayroll	public.payroll	**SELECT**	month: number, year: number	*Promise<PayrollRecord[]>*	Retrieves calculated payslips for entire workforce for specified month and year.
generatePayroll	payroll, attendance, leaves	**INSERT (Batch Calc)**	month, year	*Promise<PayrollRecord[]>*	Executes automated payroll run: computes payable days against attendance, calculates deductions and net pay.
updatePayrollStatus	public.payroll	**UPDATE**	payrollId, status	*Promise<PayrollRecord>*	Transitions payroll batch through lifecycle: Draft -> Approved -> Processed -> Paid.

9. Department & Designation Hierarchy

Source: src/lib/api/departments.ts

API / Function	Table / Target	Operation	Parameters	Return Type	Description & Business Logic
fetchDepartments	public.departments	**SELECT**	None	*Promise<Department[]>*	Retrieves all organization departments with headcount count and manager association.
createDepartment	public.departments	**INSERT**	DepartmentPayload	*Promise<Department>*	Creates new operational department in corporate directory.
updateDepartment	public.departments	**UPDATE**	id, Partial<Department>	*Promise<Department>*	Modifies department name, department head, or description.
deleteDepartment	public.departments	**DELETE**	id: string	*Promise<void>*	Deletes department if no active employees are assigned.
fetchDesignations	public.designations	**SELECT**	None	*Promise<Designation[]>*	Retrieves job titles, pay grade bands, and linked department references.
createDesignation	public.designations	**INSERT**	DesignationPayload	*Promise<Designation>*	Defines new job title and attaches it to parent department.
updateDesignation	public.designations	**UPDATE**	id, Partial<Designation>	*Promise<Designation>*	Updates designation title, minimum/maximum experience, and grade level.
deleteDesignation	public.designations	**DELETE**	id: string	*Promise<void>*	Removes job title from organization roster.

10. Document Storage & Meeting Hall Reservations

Source: src/lib/api/core.ts & modules.ts

API / Function	Table / Target	Operation	Parameters	Return Type	Description & Business Logic
fetchDocuments	public.documents	**SELECT**	employeeId?: string	*Promise<Document[]>*	Retrieves employee documents (Contracts, ID Proofs, Payslips, Educational Certs).
uploadDocument	storage.objects, documents	**STORAGE + INSERT**	file, metadata	*Promise<Document>*	Uploads binary file to Supabase storage documents bucket and inserts record in documents.
deleteDocument	storage.objects, documents	**STORAGE + DELETE**	documentId: string	*Promise<void>*	Deletes file from cloud bucket and removes database record.
fetchMeetingHallBookings	public.meeting_hall_bookings	**SELECT**	None	*Promise<Booking[]>*	Retrieves conference room and meeting hall reservation schedule.
createMeetingHallBooking	public.meeting_hall_bookings	**INSERT**	BookingPayload	*Promise<Booking>*	Reserves meeting room with start/end time, organizer, agenda, and attendee list.
updateMeetingHallBookingStatus	public.meeting_hall_bookings	**UPDATE**	bookingId, status	*Promise<Booking>*	Confirms, checks in, or cancels meeting room reservation.
deleteMeetingHallBooking	public.meeting_hall_bookings	**DELETE**	bookingId: string	*Promise<void>*	Removes reservation and frees hall on corporate calendar.

11. Task Workflow, Broadcasts & Notifications

Source: src/lib/api/core.ts & notificationService.ts

API / Function	Table / Target	Operation	Parameters	Return Type	Description & Business Logic
fetchTasks	public.tasks	**SELECT**	assignedTo?: string	*Promise<Task[]>*	Retrieves operational employee tasks with priority, due dates, and completion status.
createTask	public.tasks	**INSERT**	TaskPayload	*Promise<Task>*	Assigns new deliverable to employee with deadline and priority (Low/Med/High/Urgent).
updateTaskStatus	public.tasks	**UPDATE**	taskId, status	*Promise<Task>*	Transitions task through workflow (Todo -> In Progress -> Under Review -> Completed).
deleteTask	public.tasks	**DELETE**	taskId: string	*Promise<void>*	Removes task from board.
fetchAnnouncements	public.announcements	**SELECT**	None	*Promise<Announcement[]>*	Retrieves active company-wide circulars, notices, and HR policy broadcasts.
createAnnouncement	public.announcements	**INSERT**	AnnouncementPayload	*Promise<Announcement>*	Publishes organization-wide announcement displayed on employee dashboards.
deleteAnnouncement	public.announcements	**DELETE**	id: string	*Promise<void>*	Archives broadcast notice.
fetchNotifications	public.notifications	**SELECT**	userId: string	*Promise<Notification[]>*	Fetches in-app notification alerts for active user with read/unread flags.
markNotificationRead	public.notifications	**UPDATE**	notificationId: string	*Promise<void>*	Marks single notification banner as read.
markAllNotificationsRead	public.notifications	**UPDATE**	userId: string	*Promise<void>*	Clears all unread badge indicators for active user.

12. Performance Appraisal & KPI Goals

Source: src/lib/api/modules.ts

API / Function	Table / Target	Operation	Parameters	Return Type	Description & Business Logic
fetchPerformanceGoals	public.performance_goals	**SELECT**	employeeId?: string	*Promise<Goal[]>*	Retrieves quarterly OKRs and KPIs with target metric and completion percentage.
createGoal	public.performance_goals	**INSERT**	GoalPayload	*Promise<Goal>*	Sets new performance objective for employee with evaluation criteria.
updateGoalStatus	public.performance_goals	**UPDATE**	goalId, progress, status	*Promise<Goal>*	Updates goal progress percentage (0-100%) and completion milestone.
fetchPerformanceReviews	public.performance_reviews	**SELECT**	employeeId?: string	*Promise<Review[]>*	Retrieves annual/quarterly appraisal reviews including self-ratings and manager scores.
createReview	public.performance_reviews	**INSERT**	ReviewPayload	*Promise<Review>*	Submits finalized performance review with rating (1-5), promotion advice, and bonus notes.

13. Transactional Mail, Background Daemons & System Services

Source: src/lib/api/email.ts, vite.config.ts, Edge Functions

API / Function	Table / Target	Operation	Parameters	Return Type	Description & Business Logic
sendCandidateApplicationEmail	audit_logs, SMTP Gateway	**HTTP / QUEUE**	CandidateApplicationEmailPayload	*Promise<{ success }>*	Generates branded HTML confirmation, dispatches to local SMTP or queues in audit_logs.
POST /api/send-email	Vite Dev Server (Port 5174)	**HTTP POST (SSL/TLS)**	{ to, subject, html, refId }	*JSON: { success: true }*	Direct TLS connection to kindle.herosite.pro:465 via authenticated hr@oklut.com user.
Supabase Edge Function: send-email	Supabase Deno Deploy	**HTTP POST (v1/send-email)**	{ to, subject, html, fromName }	*JSON: { success: true }*	Cloud-native serverless transactional mail dispatcher connecting to SMTP host over TLS.
fetchDashboardStats	employees, leaves, candidates	**AGGREGATE SELECT**	None	*Promise<DashboardMetrics>*	Aggregates workforce size, active leave count, open positions, and recruitment metrics.
fetchAuditLogs	public.audit_logs	**SELECT**	None	*Promise<AuditLog[]>*	System event log recording authentication attempts, email delivery queues, and security actions.